

Reference repositories — round 1

Purpose. Ten repositories studied for lessons that apply to *this* build. Each entry names the specific gap or finding it informed, not a general “good practice”. A reference that does not change a decision was not worth the read.

Method. GitHub API metadata + README/source review. Stars are as of 2026-09-15.

A finding from the search itself, recorded first because it shaped the list. Searching `sportsbook react native`, `odds api typescript`, and `betting odds` returns almost exclusively scrapers, arbitrage bots and tout front-ends — the highest-starred domain results are `soccerapi` (179★), `bet365-scraper` (154★), `sportsbookreview-scraper` (52★) and `betting-crawler` (49★), and every React Native “sportsbook” repo returned **zero stars**. There is no credible open-source reference for a *trust-first* sports-intelligence client. That is information: it means the design and compliance decisions in this build cannot be copied from a peer, and it means the category’s open-source gravity is toward exactly the thing `docs/strategy/platform-gaps-triage.md` rejects. The list below is therefore weighted toward *engineering* references rather than domain ones.

1. `expo/expo` — 52,262★ · MIT

What it is. The framework this app is built on; SDK 57 / RN 0.86.3 / React 19.2.3.

Lesson applied. Version truth lives in `expo/bundledNativeModules.json` inside the `expo` package tarball — not in a blog post, not in a tutorial, and not in this agent’s training data. That file is what told us `expo-widgets ~57.0.19` (the official Live Activities package) and `react-native-worklets 0.10.1` exist at all. **Applied to:** the entire dependency set in `app/package.json`, and the plugin list in `app.json`.

Lesson applied (negative). `expo-template-default@<sdk>` is the other canonical reference, and it is what revealed that the app’s `babel.config.js` must agree with `app.json`’s `experiments.reactCompiler` — a disagreement between those two is a build that works locally and diverges in a prebuild.

2. `obytes/react-native-template-obytes` — 4,345★ · MIT

What it is. A production Expo starter with the quality scaffolding most projects bolt on later.

Lessons applied.

- **Envs are validated at build time, not read ad hoc.** The template fails the build on a missing `EXPO_PUBLIC_*` rather than discovering it at runtime on a customer’s phone. This app reads one env var (`EXPO_PUBLIC_API_BASE_URL`) and defaults it to the canonical www host rather than failing — a deliberate divergence, because a missing API base should point at production, not brick the app.

- **expo-router typed routes.** Enabled in `app.json` (`experiments.typedRoutes`), which is what makes `router.push({ pathname: "/pick/[id]", params: { id } })` a compile-checked call rather than a stringly-typed navigation that silently no-ops.
- **Test IDs on every interactive element.** Adopted: `testID` is a prop on every primitive in `src/components/primitives.tsx`, so the future component tests have handles without a refactor.
- **CI gates: lint → typecheck → test, in that order, cheapest first.** Adopted as the ordering of `scripts/verify.sh`.

Lesson NOT adopted. Tailwind via NativeWind. This app has a bespoke token system ported from FIELD; layering a utility framework over it would put two class vocabularies in one codebase, which is the same drift problem finding F2 describes.

3. TanStack/query — 50,297★ · MIT

What it is. Async server-state management; used as the app's data layer.

Lesson applied (and then deliberately overridden). The library's default advice is `retry: 3` with exponential backoff. This app sets `retry: 0` and puts retry in the HTTP client instead, because the client understands the *meaning* of a failure: a 503 with `code: "bootstrap"` is a gate that will not open by retrying, and a 429 carries a `Retry-After` header that a generic backoff would ignore. **Two retry layers that disagree is how an app hammers a server that already told it to stop.** Documented in `src/providers/AppProviders.tsx`.

Lesson applied. Discriminated-union return values (`ApiResponse<T>`) rather than thrown errors, because this product's failure modes are *states with their own copy* — gate, stale, rate-limited, offline. Collapsing them into one thrown `Error` destroys the copy that the server went to the trouble of distinguishing.

4. callstack/react-native-testing-library — 3,419★ · MIT

What it is. The standard component-testing library for React Native.

Lesson applied. Its guiding principle — “*the more your tests resemble the way your software is used, the more confidence they give you*” — is why the app's components carry real `accessibilityLabel` and `accessibilityRole` values rather than test-only IDs alone. The tests this enables are written against what a screen-reader user experiences, so an accessibility regression fails a test.

Gap this exposes. There are **no component tests in this build**, because the library needs a working React Native renderer and this host has none. It is recorded as gap 3 in the audit rather than quietly omitted. The four states most worth testing first, in order: stale gate, bootstrap gate, redacted confidence, emptied board.

5. EvanBacon/expo-apple-targets — 1,384★

What it is. A config plugin that generates real Apple targets (widgets, Live Activities, App Clips) outside `/ios`, preserving Continuous Native Generation.

Lesson applied. This is the missing tool for the app's Live Activity.

`plugins/withLiveActivity.js` declares the app-side support (Info.plist keys, entitlements) and deliberately does **not** author a half-target: “a half-created target fails in a way that looks like a signing problem.” When the widget target is built, this is the tool, and it requires CocoaPods $\geq 1.16.2$ and Xcode 16 — which is another reason the full build needs a Mac.

Lesson applied to scope. The plugin's existence is what justified shipping the *declaration* now and the *target* later: the switch is additive rather than a signing change.

6. `Shopify/react-native-skia` — 8,591★ · MIT

What it is. Skia graphics bindings for React Native.

Lesson considered, and rejected for v1. The calibration curve is the app's signature visualisation, and Skia would render it faster and more beautifully than `react-native-svg`. It was not used, for three reasons that are all about this specific product:

1. A reliability diagram is **four points and a diagonal**. Skia is a graphics engine; the performance argument does not exist at this size.
2. The design contract says “numerals first — the number is always visible before the chart renders”. The chart is decoration on top of numbers that are already on screen, and making it the expensive part inverts that.
3. A GPU surface is the largest dependency in the binary for the smallest return.

Where this verdict flips: the Market Gravity Meter (a 0–100 pressure visual with direction) and the Signal Ticker (continuous scroll), both named in `DESIGN.md` as signature components and both genuinely continuous. Those want Skia, and the note is here so the next round does not re-litigate it.

7. `software-mansion/react-native-reanimated` — 10,994★ · MIT

What it is. The animation library; v4.5.1 with `react-native-worklets` on this SDK.

Lesson applied (build correctness). The worklets Babel plugin **must be LAST** in the plugin list. Placed earlier it silently drops the worklet transform and animations fail at *runtime*, not at build time. Encoded as a comment in `babel.config.js` with the reason, so a future reorder does not reintroduce it.

Lesson applied (doctrine). The design contract is stricter than the library: “Never animate data values mid-display — a confidence score changing from 72 to 74 should update instantly.”

Reanimated makes animating a number trivial, which is exactly why the rule has to be enforced by policy rather than by capability. The app's motion is limited to fade and a 4px translate, and the `MotionPreferenceProvider` collapses all durations to `0.001` when Reduce Motion is on — read ONCE at the root, because the contract says “never override it in components” and one source is the only way to guarantee that.

8. `fastlane/fastlane` — 42,113★ · MIT

What it is. The incumbent iOS release automation.

Lesson applied. Fastlane’s `deliver` / `pilot` split (metadata vs TestFlight) is the same split the `asc` CLI exposes, and Fastlane’s documented ordering — **validate, then TestFlight, then review** — is the ordering encoded in `docs/store/SUBMISSION.md` §9. The `asc validate` step is there specifically because it catches the missing-privacy-policy and missing-icon class of rejection before a human reviewer sees it.

Lesson applied (divergence). Fastlane itself is not used: it is a Ruby toolchain, and the `asc-cli` skill already provides a native binary for this host. Tooling that requires a runtime the host does not have is not a shorter path.

9. `style-dictionary/style-dictionary` — 4,807★ · Apache-2.0

What it is. A build system that generates platform-specific style artefacts from ONE source of truth.

This is the direct answer to audit finding F2. The repo currently has

`apps/web/styles/design-tokens.css` (authoritative), `DESIGN.md`’s YAML front matter (stale), `apps/web/tailwind.config.ts` (partially stale), and `design-system/colors_and_type.css` (fully stale). `BRAND_AND_DESIGN_SYSTEM.md` §2 flagged the drift in June; it is still there in September, and this build hit it first-hand — the port had to choose between two files that disagree about the primary accent colour.

Lesson applied. The app ports from ONE file — `design-tokens.css`, which declares itself the sole authority — and its linter reads the token maps directly, so a token that does not exist is a build failure rather than a silently `undefined` style. That is a local fix.

Recommended upstream action. The real fix is to make the drift structurally impossible:

`design-tokens.css` (or a `.json` next to it) becomes the source, Style Dictionary generates `tailwind.config.ts` and the app’s `tokens.ts`, and a CI check fails if `DESIGN.md`’s front matter does not match. Two artefacts that are *supposed* to agree will not; one artefact plus a generator will.

10. `S1M0N38/soccerapi` — 179★

What it is. A minimal soccer odds scraper.

Why it is on the list. Not for its code — for what it represents. It is the most-starred odds-related result, it is ~200 lines of scraping, and it stores raw bookmaker prices with no provenance, no fetch timestamp, and no notion of a source’s rights.

Lesson applied (inverted). Everything the app displays carries a freshness stamp, and every pick carries a receipt hash. That is the opposite of the category norm, and it is the whole moat. The app’s `FreshnessStamp` being a *required* prop on every data card — rather than an optional nicety — is the concrete expression of this reference.

Lesson applied (legal). `apps/web/lib/scraping/` and the source-rights registry exist because the repo takes rights seriously. The app never renders raw provider payloads — the audit surface returns hashes, byte counts and fetch times, never redistributed feeds. That boundary is deliberate and is stated in the UI (`app/pick/[id].tsx` , the evidence panel).

What the ten have in common, and what that means here

Pattern	Seen in	Applied as
One source of truth, generated outputs	style-dictionary, expo	Tokens from one file; linter reads the real maps
Fail at build, not on a device	obytes, expo, reanimated	Linter + self-test; required props over conventions
Retry belongs where the <i>meaning</i> is known	TanStack	Retry in the client, <code>retry: 0</code> in the query layer
Test what a user experiences	RNTL	Real a11y labels on every component
Add native capability without leaving the managed workflow	apple-targets	Declare now, target later, additive switch
The domain category will pull you toward touts	the search itself	The withholding rules, and their tests

What round 2 should look for

Deliberately not in round 1, because they answer questions this pass did not reach:

1. **StoreKit 2 subscription state machines** — restore, grace period, billing retry, and the “purchase succeeded but the server has not heard yet” reconciliation window. This is where subscription apps actually lose money.
2. **Offline-first sync with conflict resolution** — the app is read-mostly now; the watchlist makes it a writer, and a queue without a merge rule is a queue that duplicates.
3. **Accessibility-first charting** — how to make a reliability diagram legible to VoiceOver without reading sixteen numbers.
4. **Privacy-preserving analytics** — the app currently collects product interaction unlinked; the question is whether the *value* justifies the collection at all.
5. **Detox/Maestro-class E2E** — the component-test gap (audit gap 3) has a second half, and it needs a host that can run a simulator.